

UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Departamento de Industrias

INFORME DE LABORATORIO

Laboratorio 3

Automatización de Procesos con n8n: Triage de Devoluciones

ICN 292 Sistemas de Información para la Gestión
Segundo Semestre 2026

Estudiante: Cristóbal Muñoz

RUT: 21.657.535-0

Valor de S: 535

Repositorio GitHub:

<https://github.com/IMicit/ICN292-Lab3-Munoz-Cristobal>

Profesores: José Miguel González Paul · José Luis Sáez Tamayo

Campus Vitacura
Septiembre de 2026

Resumen Ejecutivo

En este informe presento cómo armé un sistema automático usando n8n para ordenar las solicitudes de devolución de Andes Hogar SpA. A partir de mi RUT saqué mi semilla $S = 535$. Con eso, el monto de corte para mandar casos a revisión manual quedó en \$65.000 y el plazo máximo para aceptar productos quedó en 28 días corridos.

El trabajo tiene tres partes: primero, un flujo emisor que crea y manda 15 casos de prueba. Segundo, un flujo de triaje que recibe la información, la filtra siguiendo las reglas del caso, saca el valor de la UF usando una API y decide qué hacer con cada solicitud. Y tercero, un flujo que corre solo en la noche para hacer un reporte diario, al cual le agregué un arreglo matemático para que no se caiga si le toca dividir por cero. Al final, probé meterle datos con errores y números justo en el límite para comprobar que el sistema no se congela y clasifica todo bien.

1. Parámetros y Semilla Personal

Usando mi RUT y la fórmula de la guía, llegué a la semilla $S = 535$. Con esto calculé los parámetros que usé para armar las reglas del flujo:

Semilla personal

RUT: 21.657.535-0 $\implies S = 535$.

- **Umbral gerencial (U):** Es el monto tope. Si la compra lo supera, se va a revisión manual:

$$U = \$50,000 + \$1,000 \cdot (535 \text{ (mód 40)}) = \$50,000 + \$1,000 \cdot 15 = \$65,000 \quad (1)$$

- **Plazo límite (D):** Fijado por el caso en **28** días corridos. Si la solicitud se pasa de ese tiempo, se rechaza de inmediato.
- **Canal de conexión:** El flujo de triaje recibe los datos a través de peticiones HTTP POST en formato JSON usando un nodo Webhook.

Parámetro	Regla usada en n8n	Valor
Umbral U	Si el monto es mayor a U , se desvía a revisión	\$65.000
Plazo D	Si pasaron más de D días, se rechaza la solicitud	28 días
Semilla S	Número base calculado con mi RUT	535
Protocolo	Forma en que el emisor le manda los datos al triaje	POST / JSON

Cuadro 1: Parámetros calculados para el laboratorio

2. Parte A: Implementación del Triaje y Emisor

2.1 Flujo principal de Triaje

Este flujo parte con un Webhook que se queda escuchando para recibir datos. A ese nodo le conecté un Switch que funciona como un gran filtro, revisando la información en orden y mandándola por estas salidas:

1. **DATOS_INVALIDOS:** Descarta si falta el ID de la solicitud o si el monto viene en cero o negativo.
2. **RECHAZO_PLAZO:** Revisa si los días desde la compra son mayores a 28.
3. **RECHAZO_ESTADO:** Filtra si el producto viene marcado como dañado por el cliente.
4. **REVISION_MONTO:** Separa las compras que pasan el límite de los \$65.000.
5. **APROBACION:** Si la solicitud no tuvo problemas con ningún filtro anterior, cae acá por descarte.

Para los casos que se aprueban o quedan en revisión, puse un nodo HTTP Request que consulta a la API de `mindicador.cl` para indexar el valor de la UF. Al final, el flujo guarda el registro y devuelve un aviso de éxito al emisor con un código HTTP 200.

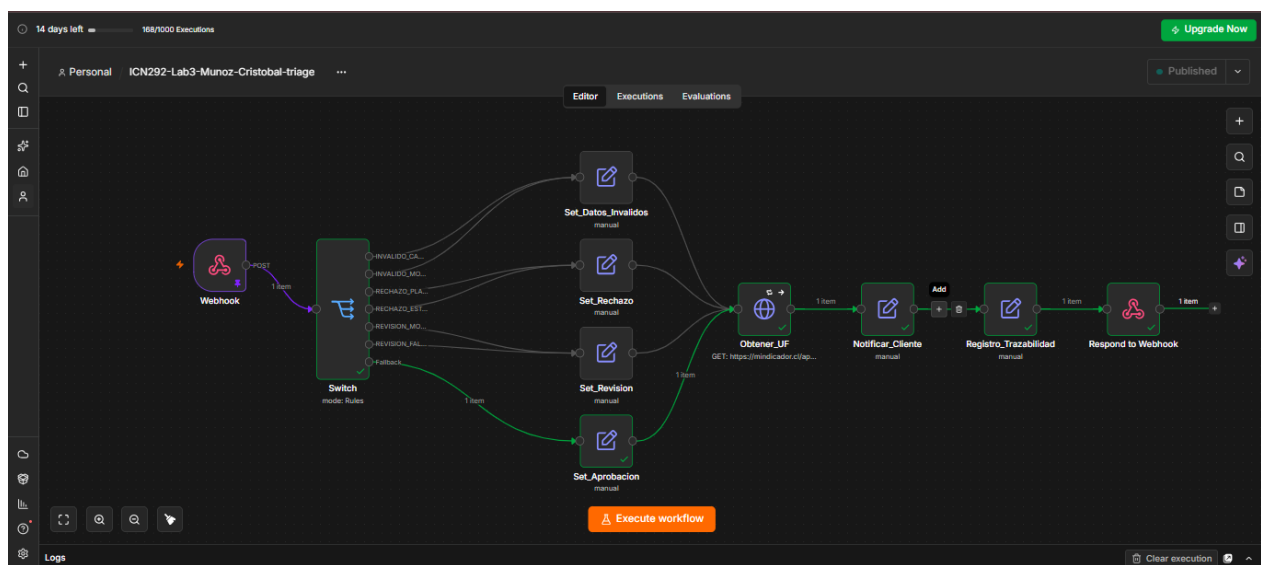


Figura 1: Flujo de Triage en n8n con el nodo Switch y la conexión a la API

2.2 Flujo Emisor

Para poder hacer las pruebas armé un segundo flujo más simple. Agregué un nodo de código en JavaScript y le pegué la lista con los 15 registros que venían en la guía (que traen distintos montos, días y estados). Después, un nodo HTTP Request los agarra y los envía de a uno al Webhook del triaje.

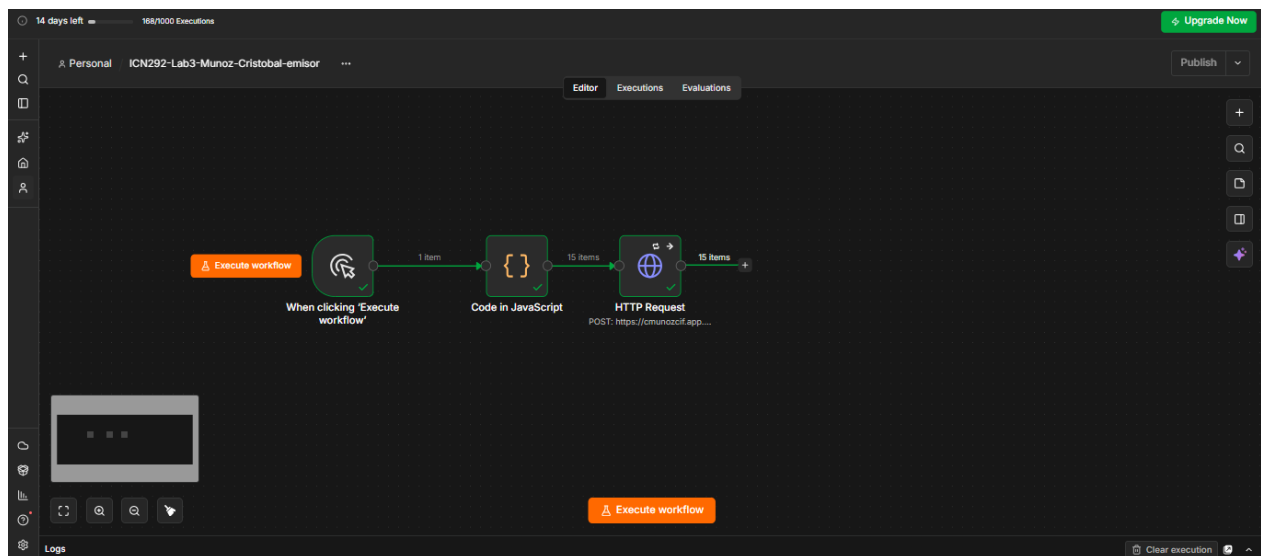


Figura 2: Tabla que muestra cómo el Triage clasificó los 15 casos que mandó el Emisor

3. Parte B: Consolidación y Reporte Diario

El tercer flujo lo hice para sacar un reporte al final del día. Empieza con un nodo Schedule Trigger para que corra solo a cierta hora. El flujo lee todas las solicitudes del día y corre un script que suma las platas y cuenta cuántos casos se fueron por cada ruta.

Para sacar el porcentaje de aprobación sin que haya errores, le puse una condición matemática en el código (usando un operador ternario). Básicamente, si en un día no entra ninguna solicitud ($total = 0$), el cálculo devuelve 0% automáticamente. Con esto me aseguro de que el programa no tire error por intentar dividir por cero.

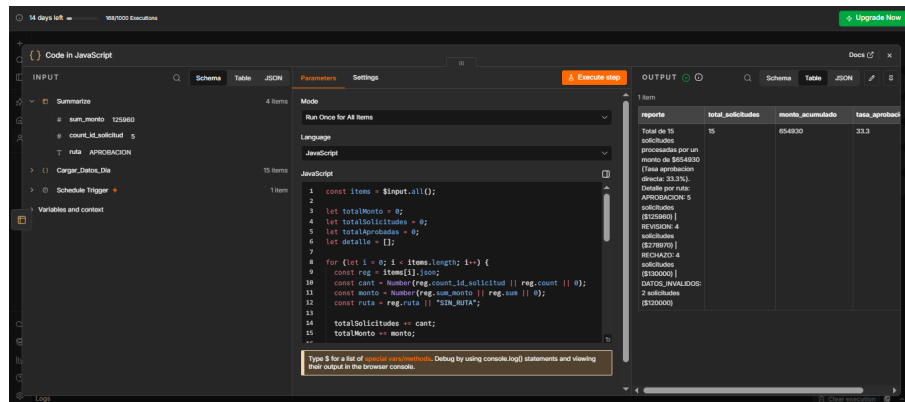


Figura 3: Flujo programado para armar el reporte diario automáticamente

4. Parte C: Manejo de Errores y Pruebas de Borde

4.1 Pruebas con datos malos

Para ver si el sistema aguantaba bien, le mandé tres casos con datos malos a propósito:

Caso	Dato ingresado	Lo que debió pasar	Lo que pasó
Entrada 1 (Fallo servidor)	Monto negativo (−5000) y el nodo no estaba preparado	Desviarse por las reglas del negocio	El flujo remoto se cayó y tiró un error HTTP 500.
Entrada 2 (Monto en cero)	Monto igual a 0 (DEV-ERR-02)	Irse a datos inválidos	Todo bien (HTTP 200), se fue a DATOS_INVALIDOS.
Entrada 3 (Monto negativo)	Monto igual a -1 (DEV-ERR-03)	Irse a datos inválidos	Todo bien (HTTP 200), se fue a DATOS_INVALIDOS.

Cuadro 2: Resultados al enviar datos problemáticos al sistema

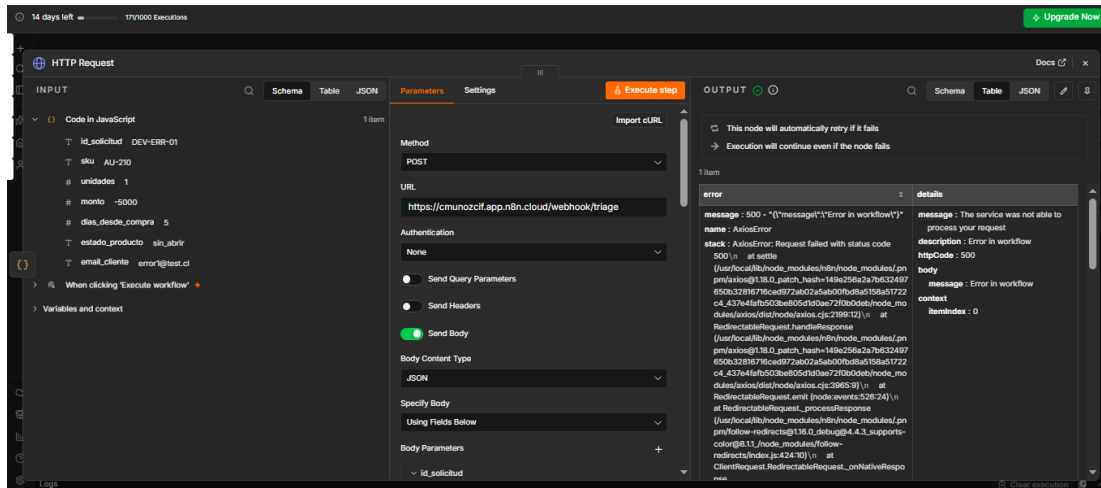


Figura 4: Entrada 1: Error HTTP 500 porque el flujo remoto no supo qué hacer con el dato

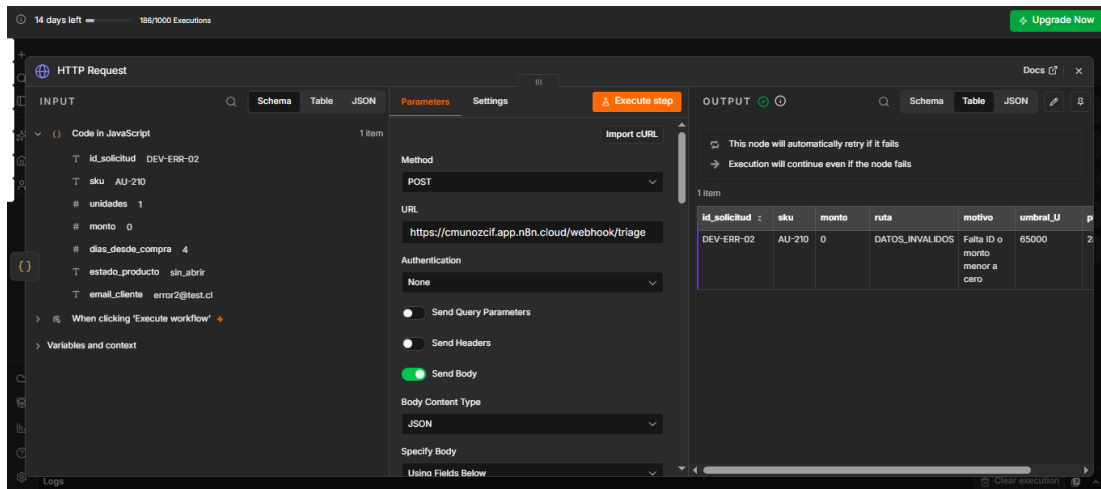


Figura 5: Entrada 2: El sistema filtró bien la solicitud que traía monto cero

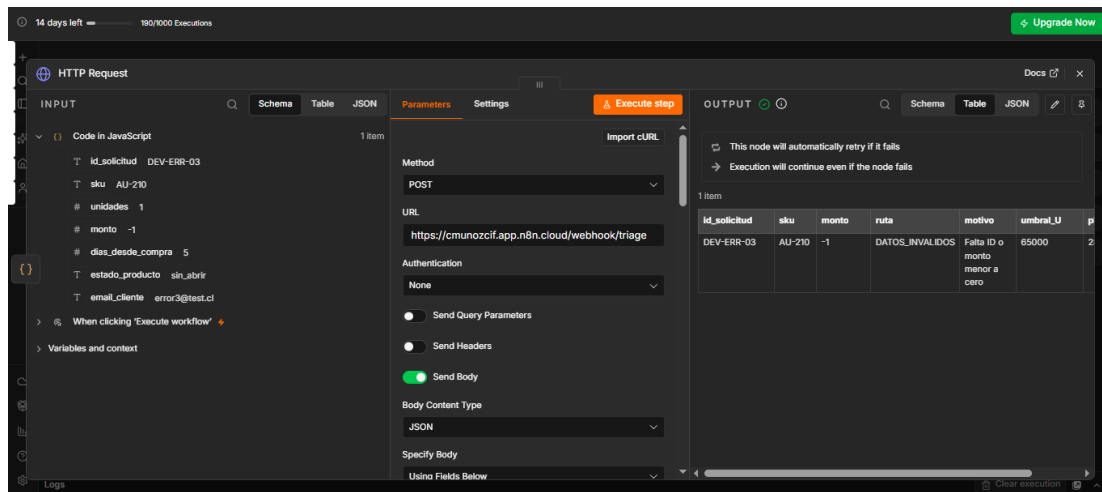


Figura 6: Entrada 3: El sistema capturó correctamente el monto negativo

4.2 Uso de On Error en n8n

Cuando uno configura los nodos en n8n, hay una pestaña llamada *Settings* donde se le dice al programa qué hacer si hay un error:

- **Stop Workflow:** Es la opción por defecto. Si ocurre un error, el flujo se detiene de inmediato y el nodo queda marcado en rojo.
- **Continue Regular Output:** El flujo no se para, sino que le manda el detalle del error al siguiente nodo como si fuera un dato más.
- **Continue Error Output:** Abre una salida extra en el nodo (de color rojo) para mandar por ahí solo los casos que fallaron y procesarlos aparte.

En mi sistema le puse *Continue Regular Output* al nodo que consulta la UF. Así, si la página de los indicadores económicos se cae, el flujo sigue funcionando igual y no detiene el proceso de las devoluciones. Por el contrario, en el nodo Switch dejé la opción *Stop Workflow*, porque si llega a fallar una regla es mejor que pare todo antes de que clasifique mal los datos.

4.3 Para qué sirven los Error Workflows

Un Error Workflow es un flujo que funciona como un respaldo. Se activa solo (usando un nodo que se llama *Error Trigger*) cuando cualquier otro flujo de la plataforma falla.

Esto sirve mucho en procesos como mi reporte diario. Como ese flujo funciona solo en la noche, si algo falla nadie se entera. Si no ponemos un Error Workflow, el error pasa colado. Pero si lo conectamos, el sistema te puede mandar un correo o un mensaje avisando al tiro que hubo un problema.

4.4 Pruebas justo en los límites

Hice dos pruebas mandando exactamente los números del límite para ver por qué rama los tiraba el Switch:

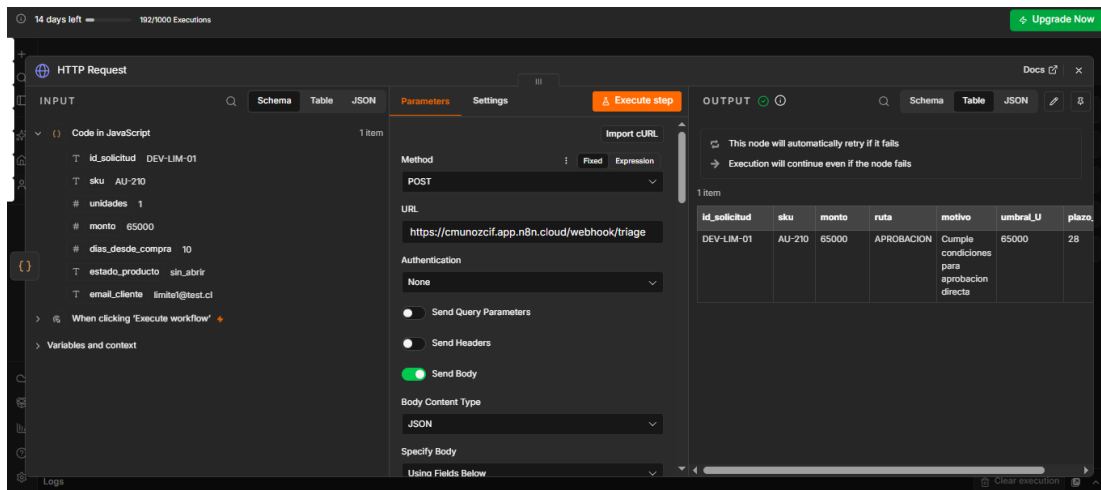


Figura 7: Prueba A: Solicitud enviada justo con \$65.000

- **Límite A (\$65.000):** Mandé el caso DEV-LIM-01 con exactamente \$65.000 y se fue directo a APROBACION. Esto pasó porque la regla dice que el monto tiene que ser estrictamente mayor a 65.000 (>) para irse a revisión. Como era igual, se aprueba.

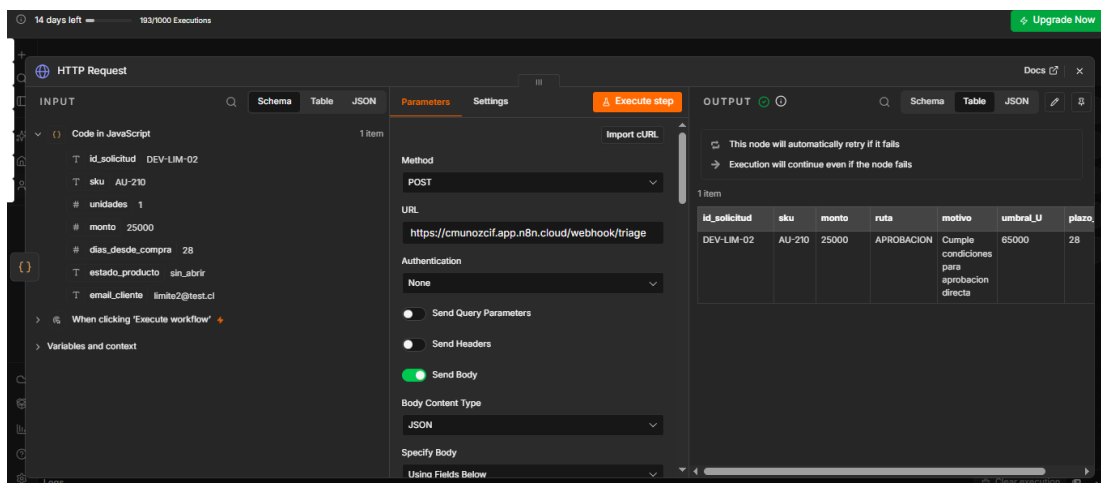


Figura 8: Prueba B: Solicitud enviada con exactamente 28 días desde la compra

- **Límite B (28 días):** Mandé el caso DEV-LIM-02 con 28 días justos. También se fue a APROBACION porque la regla rechaza solo los casos que tienen más de 28 días (>). El día 28 todavía está dentro del plazo.

4.5 Error por tipo de dato

Para probar qué pasa cuando se fuerza un tipo de dato incorrecto, cambié en el Switch la regla del monto a formato de texto (String) y mandé la solicitud DEV-C01 con un monto de \$9.990.

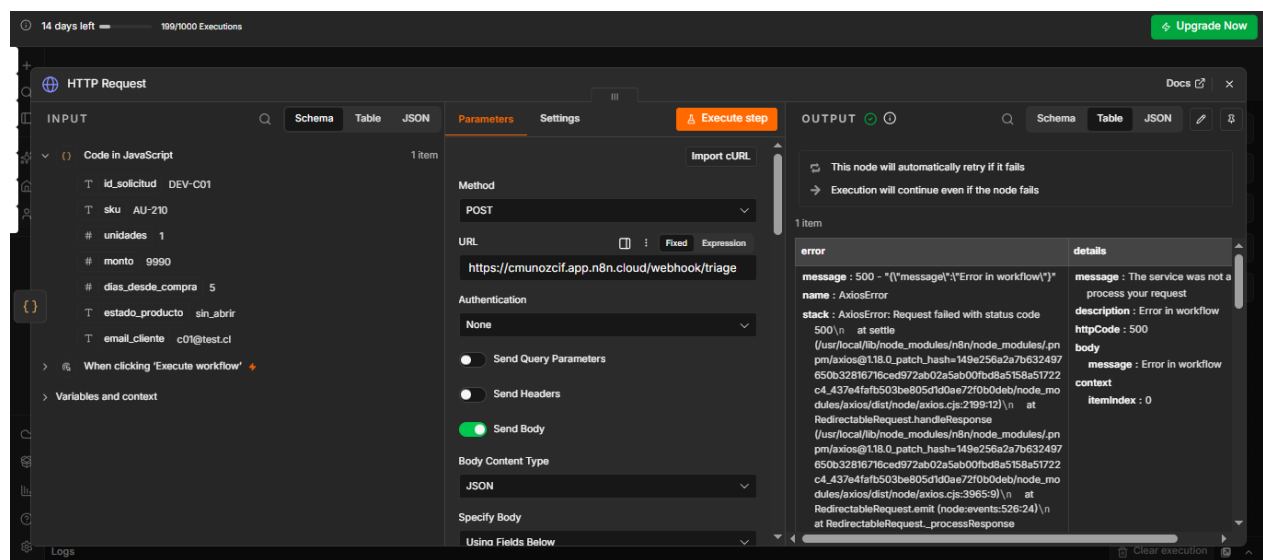


Figura 9: Error HTTP 500 provocado en el Webhook al evaluar el monto como texto en el Switch

¿Por qué pasó esto?: Al configurar la regla del Switch esperando una comparación numérica pero forzando el campo a tipo texto, el motor de n8n no pudo resolver la operación internamente. Esto provocó que el flujo remoto de triaje colapsara y respondiera con un error HTTP 500 (Error in workflow), demostrando que una mala definición en los tipos de datos bota la ejecución completa en lugar de clasificar la solicitud.

5. Conclusiones

Hacer este laboratorio me sirvió harto para ver en la práctica cómo automatizar cosas que siempre se hacen a mano. Con las pruebas me di cuenta de lo importante que es decirle bien al sistema si un dato es un número o un texto, porque si uno se equivoca en eso, el programa compara mal y clasifica todo al revés. También comprobé que es súper necesario configurar qué hacer si se cae una conexión externa (como consultar la UF), para que el proceso no se congele por completo y pueda seguir atendiendo las demás solicitudes sin problema.